
Scenario 0 - SriToysRus Ltd

SriToysRus Ltd (STRU) is a Sri Lankan based company that specialises in promoting toys to children below 12 years of age. The company has 15 branches across the island and sells toys that are either imported or manufactured locally. Due to the high demand in sales within Colombo, the company has already planned to open several new branches in a few major cities. The management of the company is interested in extending the reach of the business using Internet technology. They are interested in exposing the overall inventory to the public using an Internet-based application. Some of the key functionalities expected out of this system are listed below.

- Users should be able to search for toys available for sale through the system.
- Detailed descriptions of the selected toys such as category, brand, country of origin, price, discounts, age limits, educational aspects, etc., should be clearly displayed.
- The users should be able to create personalised accounts to register and login to in the system.
- The personalised account/profile will assist to have a personalised view/history of the system - E.g. past purchases, wish list, etc. Users should be able to register their credit cards and perform purchases online.
- The purchases done online through the STRU platform will be delivered to the users' registered addresses. The delivery of the purchased goods to the users will be done by an existing delivery company that already has a platform to track and monitor the overall distribution process.
- A centralised inventory control system should be maintained so that the sale of products either through the online platform or the outlets reflects the correct view of available stock. A user should be able to repetitively select and add products to an online cart that could be later checked out through a single submission. The payment aspect using the credit card should be implemented in a secure manner by integrating it to an existing payment gateway.

Note:

- a. STRU already has a computerised inventory management system that links all the distributed outlets. This can be reused in the proposed online platform.
- b. The solution should allow further improvements to include order delivery through a delivery force that would carry a PDA-based solution to accept credit cards and cash.

Exercise

1. Propose an architecture to realise the online platform in the SriToysRus Ltd (STRU) scenario given above. a. Identify the layers/tiers of the architecture. b. List the components of each layer/tier. c. Sketch / draw the overall architecture with clear labels for tiers and components.
2. List out and briefly describe the architectural patterns used in the proposed architecture.

3. Based on the description of the above scenario, identify a legacy system that needs to be included in the architecture and discuss a method to integrate the legacy system into the overall solution.
4. List high-level sub-systems of the proposed architecture.
5. Identify and describe an interaction/integration in the architecture where Asynchronous Communication is more suitable than Synchronous communication.

Answer for Scenario 0: SriToysRus Ltd 🧸

1. Proposed Architecture

A **Microservices Architecture** is proposed to build a scalable and maintainable online platform. This architecture will decouple major business functionalities into independent services.

a. Layers/Tiers of the Architecture:

1. **Client Tier:** The user-facing applications.
2. **API Gateway Tier:** The single, secure entry point for all client requests.
3. **Business Services Tier:** A collection of independent microservices handling specific business domains.
4. **Data & Integration Tier:** Manages data storage and communication with external/legacy systems.

b. Components of Each Layer/Tier:

- **Client Tier:**
 - Web Application (Customer-facing e-commerce site)
 - (Future) Mobile Application (iOS/Android)
- **API Gateway Tier:**
 - **API Gateway:** Manages routing, authentication, and acts as a facade for the backend services.
 - **Authentication Service:** Handles user registration and login.
- **Business Services Tier:**
 - **Product Service:** Manages the toy catalog, details, and search functionality.
 - **Order Service:** Manages the shopping cart, checkout process, and order history.
 - **User Service:** Manages customer profiles, addresses, and wish lists.
 - **Inventory Service:** Manages stock levels and synchronizes with the legacy system.

- **Data & Integration Tier:**

- **Databases:** Separate databases for each service (e.g., Product DB, Order DB, User DB) to ensure loose coupling.
- **Legacy Inventory Adapter:** A specific service that wraps the existing inventory system, exposing its data via a modern API.
- **Payment Gateway Integrator:** A component to communicate securely with the external payment processor.
- **Delivery Company Integrator:** A component to communicate with the delivery company's tracking platform.

c. Brief Description of Architecture:

A customer using the web application interacts with a single endpoint: the API Gateway. The gateway authenticates the user and routes their requests to the appropriate microservice. For instance, Browse for a toy hits the Product Service. Adding it to the cart involves the Order Service. When the customer checks out, the Order Service coordinates with the Inventory Service (which talks to the legacy system via its adapter), the Payment Gateway Integrator, and the Delivery Company Integrator to complete the purchase.

2. Architectural Patterns

- **Microservices Architecture:** The system is decomposed into fine-grained, independent services (Product, Order, etc.). This allows teams to develop, deploy, and scale services independently.
- **API Gateway Pattern:** All client requests are routed through a single gateway. This simplifies the client code and centralizes cross-cutting concerns like security, monitoring, and rate limiting.
- **Adapter Pattern:** Used to "wrap" the legacy inventory system and the third-party delivery company API. This pattern translates their proprietary interfaces into a standardized interface that the rest of our application can easily use.

3. Legacy System Integration

- **Legacy System Identified:** The "computerised inventory management system that links all the distributed outlets" is the legacy system.
- **Method of Integration:** A hybrid approach using an **Adapter** and a **Message Broker** is recommended.
 1. **Real-time Stock Check (Synchronous):** An **Adapter** service will be built. This service exposes a simple REST API (e.g., GET /stock/{product_id}). When a user views a product page, the platform calls this adapter, which in turn queries the legacy system in real-time to show the most accurate stock level.
 2. **Inventory Updates (Asynchronous):** When an online sale is confirmed, the **Order Service** publishes a message like ProductSold to a **Message Broker** (e.g., RabbitMQ). The **Inventory Service** subscribes to this message and updates the legacy system. This

ensures the checkout process is fast for the user and resilient; if the legacy system is slow to respond, it doesn't block the sale.

4. High-Level Sub-Systems

- **Product Catalog Management:** Handles all toy information, categories, and search.
- **User Account Management:** Manages customer profiles, authentication, and purchase history.
- **Order & Checkout System:** Manages the shopping cart, payment processing, and order finalization.
- **Inventory Management System:** The new service layer responsible for synchronizing stock between the online platform and the physical outlets.
- **Third-Party Integration System:** A logical grouping of components that integrate with the payment gateway and delivery company.

5. Asynchronous Communication Scenario

- **Interaction/Integration: Order Confirmation and Post-Purchase Operations.** After a user successfully pays for their order, several things need to happen: the inventory must be updated, a confirmation email must be sent, and the delivery must be scheduled with the external delivery company.
- **Justification:** Performing all these tasks synchronously would force the user to wait on the "Processing..." screen for a long time, leading to a poor experience. An asynchronous flow is much better:
 1. The **Order Service** receives a successful payment confirmation.
 2. It immediately returns a "Thank You" page to the user.
 3. Simultaneously, it publishes an OrderPlaced event to a message broker.
 4. Three different services subscribe to this event and act in parallel:
 - The **Inventory Service** decrements the stock in the legacy system.
 - A **Notification Service** sends a confirmation email to the user.
 - The **Delivery Integration Service** sends the order details to the delivery company's API. This approach is **faster** for the user, more **resilient** (if email fails, the order is still valid), and **scalable**.

Scenario 1: HealthConnect Patient Portal

Scenario:

HealthConnect is a large private hospital group with several branches across Sri Lanka. Each hospital currently operates its own independent Patient Information System (PIS), leading to fragmented patient records. HealthConnect wants to launch a unified web and mobile platform for its patients. The goal is to provide a single point of access for patients to view their consolidated medical history, book appointments, view lab reports, and pay bills. The system must be secure, reliable, and capable of handling sensitive patient data in compliance with health regulations.

Note:

- a. The existing PIS at each hospital are legacy systems with varying database structures and communication protocols. They are critical and cannot be replaced in the short term.
- b. The new platform should be designed to incorporate a future telemedicine (video consultation) feature.

Exercise:

1. Propose an architecture to realize the HealthConnect platform. a. Identify the layers/tiers of the architecture. b. List the components of each layer/tier. c. Briefly describe the proposed architecture.
2. List and briefly describe two architectural patterns that would be suitable for this architecture.
3. Identify the primary legacy system challenge and propose a specific method to integrate these systems into the overall solution.
4. List four high-level sub-systems of the proposed architecture.
5. Identify and describe an interaction in the architecture where Asynchronous Communication is more suitable than Synchronous communication.

Answers for Scenario 1: HealthConnect

1. Proposed Architecture

A **Service-Oriented Architecture (SOA)** or a **Microservices Architecture** would be the most suitable approach. This allows for decoupling services and integrating disparate legacy systems effectively.

a. **Layers/Tiers of the Architecture:**

- **Client Tier:** The user-facing applications.
- **Presentation Tier / API Gateway:** Manages and secures access to the backend services.
- **Business/Service Tier:** Contains the core business logic, broken down into individual services.

- **Integration/Data Tier:** Manages communication with the legacy systems and the new platform's databases.

b. Components of Each Layer/Tier:

- **Client Tier:**
 - Web Application (for browsers)
 - Mobile Application (iOS/Android)
- **Presentation Tier / API Gateway:**
 - **API Gateway:** A single entry point for all client requests. It handles routing, authentication, and rate limiting.
 - **Authentication Service:** Manages user login and security tokens (e.g., JWT).
- **Business/Service Tier (as Microservices):**
 - **Patient Profile Service:** Manages unified patient data.
 - **Appointment Service:** Handles booking, rescheduling, and cancellation.
 - **Billing Service:** Integrates with payment gateways and generates bills.
 - **Lab Report Service:** Manages and displays diagnostic reports.
 - **Notification Service:** Sends emails, SMS, or push notifications.
- **Integration/Data Tier:**
 - **EHR Integration Service (Facade/Adapter):** A dedicated service to communicate with the various legacy PIS systems.
 - **Message Broker (e.g., RabbitMQ, Kafka):** Facilitates asynchronous communication between services.
 - **HealthConnect Database:** A new database to store data for the unified platform (e.g., user accounts, appointment schedules).

c. Brief Description:

Users interact with the web or mobile app. All requests go through the **API Gateway**, which provides a secure and unified interface. The gateway routes requests to the appropriate microservice (e.g., Appointment Service). For patient data, the **EHR Integration Service** acts as a middleman. It queries the old PIS systems at different hospitals, transforms their data into a standard format, and returns a consolidated view to the Patient Profile Service. A **Message Broker** is used for tasks that don't need an immediate response, like notifying a patient that a lab report is ready.

2. Architectural Patterns

- **API Gateway Pattern:** This pattern provides a single, unified entry point for a group of microservices. For HealthConnect, it simplifies the client applications, as they don't need to

know the addresses of individual services. It also centralizes concerns like security, monitoring, and request shaping.

- **Adapter (or Wrapper) Pattern:** This pattern is crucial for legacy system integration. An adapter is created for each legacy PIS. This adapter "wraps" the old system, translating its proprietary protocol and data format into a modern, standardized one (e.g., a REST API with JSON). The rest of the HealthConnect platform interacts with this clean interface, not the complex legacy system itself.

3. Legacy System Integration

The primary legacy challenge is the **heterogeneous nature of the existing Patient Information Systems (PIS)** at each hospital. They have different databases, data schemas, and likely no modern APIs.

Method of Integration:

The most effective method is to use the Adapter Pattern combined with an Enterprise Service Bus (ESB) or a Message Broker.

1. **Create an Adapter for each PIS:** A small software component (an adapter) will be built for each hospital's PIS.
2. **Define a Canonical Data Model:** A standard format for patient data (e.g., a standard JSON structure for "Patient Record") is defined for the entire HealthConnect platform.
3. **Translate Data:** The adapter's job is to receive a request in the standard format (e.g., "get patient history for ID 123"), translate it into a query the legacy PIS understands, fetch the data, and then transform that data from the legacy format back into the standard canonical format before returning it. This isolates the new system from the complexity of the old ones.

4. High-Level Sub-Systems

1. **User & Profile Management:** Handles patient registration, authentication, and the unified patient profile.
2. **Appointment Scheduling System:** Manages all aspects of booking and viewing appointments.
3. **Medical Records Management:** Responsible for fetching, consolidating, and displaying patient history, lab reports, and prescriptions from legacy systems.
4. **Billing and Payments System:** Manages bill generation, payment processing, and viewing payment history.

5. Asynchronous vs. Synchronous Communication

An ideal scenario for **Asynchronous Communication** is the **notification of lab report availability**.

- **Interaction:** A lab technician uploads a patient's new lab report to their hospital's PIS. The integration service detects this new report. Instead of making the patient wait or constantly poll for the report, the system can work asynchronously.
- **Process:**

1. The Lab Report Service publishes an event like LabReportReady to a message topic on the Message Broker.
 2. The Notification Service, which is subscribed to this topic, receives the event.
 3. The Notification Service then sends an email, SMS, or push notification to the patient (e.g., "Dear Jane, your blood test report is now available on the HealthConnect portal.").
- **Why it's better:** This is more efficient and provides a better user experience. A synchronous approach would require the user's app to constantly ask the server, "Is the report ready yet?", which is wasteful. The asynchronous, event-driven approach is scalable and resilient; if the notification service is temporarily down, the message remains in the queue and can be processed when the service comes back online.
-

Scenario 2: MetroLink Transit System

Scenario:

The Colombo Municipal Council wants to launch "MetroLink," a real-time public transport information system. The system aims to provide commuters with live bus tracking, accurate arrival time predictions, trip planning, and a digital wallet for fare payments. The bus fleet is already equipped with GPS trackers that periodically send location data. The council also has a legacy system that manages bus routes, schedules, and fare stages, which is currently maintained manually in a simple database.

Note:

- a. The existing bus route and fare information system is a legacy database with no direct API.
- b. The system must be highly available and provide real-time updates to a large number of concurrent users via a mobile app.

Exercise:

1. Propose an architecture for the MetroLink system. a. Identify the layers/tiers of the architecture. b. List the components of each layer/tier. c. Briefly describe the proposed architecture.
2. List and briefly describe two architectural patterns that would be suitable for this architecture.
3. Identify a key legacy system and propose a method to integrate it into the overall solution.
4. List four high-level sub-systems of the proposed architecture.
5. Identify and describe an interaction in the architecture where Asynchronous Communication is essential for the system's core functionality.

Answers for Scenario 2: MetroLink 🚌

1. Proposed Architecture

An **Event-Driven Architecture** combined with microservices is ideal for handling the real-time data stream from the GPS trackers.

a. **Layers/Tiers of the Architecture:**

- **Client Tier:** The commuter-facing mobile application.
- **API Gateway Tier:** Manages and secures API access.
- **Real-time Processing Tier:** Ingests and processes high-volume GPS data.
- **Service Tier:** Provides core business logic for trip planning, payments, etc.
- **Data Tier:** Stores static, processed, and user data.

b. Components of Each Layer/Tier:

- **Client Tier:**
 - MetroLink Mobile App (iOS/Android)
- **API Gateway Tier:**
 - **API Gateway:** Routes standard requests (e.g., login, trip planning) to the Service Tier.
 - **WebSocket Gateway:** Manages persistent, real-time connections with mobile clients for broadcasting live bus locations.
- **Real-time Processing Tier:**
 - **Data Ingestor:** A highly scalable endpoint that receives raw GPS data from buses.
 - **Message Broker (e.g., Apache Kafka):** A high-throughput message queue to handle the stream of GPS events.
 - **Location Processor Service:** Consumes GPS data from the broker, cleans it, and maps it to a specific bus and route.
- **Service Tier:**
 - **Trip Planner Service:** Calculates routes and estimates arrival times using both static schedule data and real-time location data.
 - **User & Wallet Service:** Manages user accounts and digital wallets.
 - **Notification Service:** Sends alerts for delays or service changes.
- **Data Tier:**
 - **Legacy Route Database Adapter:** A wrapper around the old database.
 - **Real-time Location Cache (e.g., Redis):** Stores the most recent location of every bus for fast lookups.
 - **Main Application Database:** Stores user data, routes, and transaction history.

c. Brief Description:

GPS trackers on buses continuously send location data to the **Data Ingestor**, which pushes these events into a **Kafka** topic. The **Location Processor Service** consumes these events, determines the bus's current position, and updates its location in the **Redis cache**. When a user opens the mobile app, it establishes a WebSocket connection via the **WebSocket Gateway** to receive a live feed of bus locations for their selected route. For trip planning, the app sends a request to the **API Gateway**, which forwards it to the **Trip Planner Service**. This service combines static route data (from the legacy DB) and real-time bus locations (from Redis) to provide an accurate Estimated Time of Arrival (ETA).

2. Architectural Patterns

- **Publish-Subscribe (Pub/Sub) Pattern:** This is the core of the real-time system. The GPS trackers are "publishers" of location data. The Location Processor Service is a "subscriber" that processes this data. The mobile apps can also act as subscribers to receive location updates. A message broker like Kafka implements this pattern, decoupling the data producers (buses) from the data consumers (backend services and clients).
- **CQRS (Command Query Responsibility Segregation):** This pattern separates the models for updating data ("Commands") from the models for reading data ("Queries"). In this scenario, the "write" side is the high-volume ingestion of GPS data. The "read" side is querying the current location of a bus. Using a highly optimized cache like Redis for reads and a robust database for writes allows the system to handle both workloads efficiently without compromise.

3. Legacy System Integration

The key legacy system is the **database managing static bus routes, schedules, and fare stages**.

Method of Integration:

A Data Synchronization and Caching approach would be best.

1. **Create a Synchronization Service:** A background service will be created. Its only job is to periodically connect to the legacy database.
2. **Extract, Transform, Load (ETL):** This service will extract all the route and fare data, transform it into a modern, easy-to-use format (like JSON documents), and load it into the new **Main Application Database** and/or a fast-access cache.
3. **Decoupling:** This process runs independently (e.g., every night or every hour). The real-time MetroLink system **never queries the legacy database directly**. It only ever reads the clean, cached copy. This protects the core application from the legacy system's poor performance or potential downtime.

4. High-Level Sub-Systems

1. **Live Vehicle Tracking System:** Ingests, processes, and broadcasts real-time GPS data.
2. **Trip Planning & ETA Engine:** Calculates routes and predicts arrival times.
3. **Fare & Wallet Management System:** Handles user digital wallets, fare calculation, and payment transactions.
4. **User Management & Notification System:** Manages user accounts and sends relevant alerts.

5. Asynchronous vs. Synchronous Communication

Asynchronous Communication is absolutely essential for **processing and distributing the live GPS location data**.

- **Interaction:** A bus sends its GPS coordinates to the MetroLink backend.
- **Process:**

1. The bus's GPS unit sends a small data packet (a "location update" event) to the Data Ingestor endpoint. It does not wait for a response. This is a classic "fire-and-forget" interaction.
 2. The ingestor immediately places this event onto a Kafka topic (asynchronous buffering).
 3. Multiple services can then subscribe to and react to this event independently. The Location Processor subscribes to update the cache. A data analytics service might subscribe to analyze traffic patterns.
- **Why it's essential:** A synchronous approach is impossible here. The system cannot afford to create a persistent connection to every single bus and process its request sequentially. The sheer volume of data would overwhelm the server. Asynchronous processing via a message broker allows the system to ingest a massive stream of events, buffer them, and process them in a controlled, scalable, and parallel manner, which is the only way to build a functional real-time tracking system for a large fleet.
-

Scenario 3: UniPortal University Management System

Scenario:

A major university in Sri Lanka wants to consolidate its aging, separate systems for student admissions, course registration, library management, and finance into a single, modern platform called "UniPortal." Currently, student data is duplicated across these systems, leading to inconsistencies. The new platform must provide a unified web portal for students to manage their entire academic lifecycle, from application to graduation. The system needs to handle extremely high traffic, especially during the course registration period at the beginning of each semester.

Note:

- a. The existing Admissions, Library, and Finance systems are legacy applications, each with its own database and proprietary interface. They cannot be replaced immediately.
- b. The solution should be designed to allow for the future addition of a full-featured e-learning module.

Exercise:

1. Propose an architecture to realize the UniPortal platform.
2. List and briefly describe the architectural patterns used in the proposed architecture.
3. Based on the description, identify the legacy systems and discuss a method to integrate them.
4. List the high-level sub-systems of the proposed architecture.
5. Identify and describe an interaction/integration in the architecture where Asynchronous Communication is more suitable than Synchronous communication.

Answer for Scenario 2: UniPortal 🎓

1. Proposed Architecture for UniPortal

a. Layers/Tiers of the Architecture:

1. **Presentation Layer:** The user-facing web portal.
2. **Service Layer (API Gateway):** A unified entry point for all frontend requests.
3. **Business Logic Layer (Microservices):** Core functionalities isolated into independent services.
4. **Integration Layer:** Contains components (adapters) that communicate with external and legacy systems.
5. **Data Access Layer:** Manages persistence for the new platform's data.

b. Components of Each Layer/Tier:

- **Presentation Layer:**
 - Student Web Portal (Built with a modern framework like React or Angular)
 - Admin/Staff Web Portal
- **Service Layer (API Gateway):**
 - **API Gateway:** Routes incoming HTTP requests to the appropriate microservice.
 - **Authentication & Session Management:** Handles user login, security tokens (JWT), and session state.
 - **Rate Limiting:** Protects services from being overwhelmed during peak loads (e.g., registration).
- **Business Logic Layer (Microservices):**
 - **Student Profile Service:** Manages a unified student identity.
 - **Course & Enrollment Service:** Handles course catalogs and student registrations.
 - **Grades Service:** Manages submission and viewing of academic grades.
 - **Notification Service:** Sends emails or SMS for important events.
- **Integration Layer:**
 - **Admissions System Adapter:** A wrapper that exposes the legacy admissions system via a RESTful API.
 - **Library System Adapter:** A wrapper for the legacy library system (e.g., for checking borrowed books).
 - **Finance System Adapter:** A wrapper for the legacy finance system to fetch fee information.
- **Data Access Layer:**
 - **UniPortal Main Database:** Stores data owned by the new platform, like user accounts and course information.
 - **Data Caches (e.g., Redis):** Caches frequently accessed data, like the course catalog, to improve performance.

c. Overall Architecture Description:

Students access the UniPortal via a web browser. All requests are first handled by the API Gateway, which manages security and routing. For actions like course registration, the gateway forwards the request to the Course & Enrollment Service. If this service needs legacy data, such as a student's financial clearance, it calls the Finance System Adapter in the Integration Layer. This adapter translates the modern API call into a format the old finance system can understand. The use of microservices

allows each piece of business logic (Profiles, Courses, Grades) to be developed, deployed, and scaled independently.

2. Architectural Patterns Used

- **Microservices Architecture:** The core business logic is broken down into small, independent services (Student Profile, Course Enrollment, etc.)¹. This improves modularity, scalability, and maintainability.
- **API Gateway Pattern:** Provides a single entry point for all client requests, simplifying the client application and centralizing cross-cutting concerns like authentication, logging, and rate limiting².
- **Adapter Pattern (Wrapper):** This pattern is used in the Integration Layer. Each legacy system (Admissions, Library, Finance) is "wrapped" with an adapter that presents a modern, clean API to the rest of the UniPortal system, hiding the complexity of the legacy integration.
- **CQRS (Command Query Responsibility Segregation):** Can be used for the course registration system. The "Command" side handles the high volume of registration requests, while the "Query" side handles requests to view available seats, allowing each path to be optimized independently.

3. Legacy System Integration

- **Legacy Systems Identified:** The existing **Admissions System, Library Management System, and Finance System** are the legacy components.
- **Method of Integration:** The most suitable method is creating **API Wrappers (Adapters)** for each legacy system, orchestrated via a central platform.
 1. **Define a Canonical Model:** A standardized data format for "Student," "FeeStatus," etc., is created for the UniPortal platform.
 2. **Build Adapters:** A separate microservice (an adapter) is built for each legacy system. For example, the FinanceAdapter will expose a simple REST endpoint like GET /students/{id}/fee-status.
 3. **Translate and Mediate:** When called, this adapter will connect to the legacy finance database/system, run the necessary queries, retrieve the data in its old format, and translate it into the canonical JSON format before sending it back.
 4. **Decoupling:** This ensures the new microservices (like the Course Enrollment Service) do not need to know any details about the legacy systems. They only interact with the modern, clean APIs provided by the adapters.

4. High-Level Sub-Systems

- **Student Lifecycle Management:** Handles admissions, profile information, and alumni status.
- **Academic Management System:** Manages courses, enrollments, timetables, and grades.

- **Financial Services System:** Integrates with the legacy finance system for fee payments and financial status checks.
- **Library Services Integration:** Integrates with the legacy library system for searches and user account information.
- **User Notification System:** Manages all outbound communications (email, SMS).
- **Authentication & Authorization System:** Manages access control for the entire platform.

5. Asynchronous Communication Scenario

- **Interaction/Integration: Processing Final Grade Submissions.** When a lecturer finalizes and submits grades for a course, several downstream actions need to occur: notifying students, updating official transcripts, and calculating the semester GPA.
 - **Justification:** Performing all these actions synchronously would make the lecturer wait for a long time after clicking "Submit." An asynchronous, event-driven approach is far superior.
 1. **Event Publishing:** When the lecturer submits grades, the **Grades Service** validates them and then publishes an event like GradesFinalized to a **Message Broker (e.g., RabbitMQ)**³³³. The UI can then immediately respond to the lecturer that the submission was successful.
 2. **Event Consumption:** Multiple services "listen" for this event:
 - The **Notification Service** consumes the event and sends emails to all students in the class.
 - The **Student Profile Service** consumes the event, recalculates the student's GPA, and updates their academic record.
 - **Resilience & Scalability:** This decouples the services. If the Notification Service is down, the grade submission still succeeds, and notifications can be sent later when the service recovers. This approach scales well, as processing can be distributed across many consumers.
-

Scenario 4: LankaPost Express Courier System

Scenario:

LankaPost, the national postal service of Sri Lanka, wants to modernize its parcel delivery service to compete with private couriers. They plan to launch "LankaPost Express," a new platform that will provide customers with real-time parcel tracking, delivery scheduling, and digital proof-of-delivery. Delivery agents will be equipped with a mobile app to update the status of each parcel (e.g., "Picked Up," "In Transit," "Delivered") by scanning a QR code on the package. This status needs to be reflected instantly on the customer-facing web portal and mobile app.

Note:

- a. LankaPost has an existing internal "Mailsorting System" that assigns parcels to delivery routes. This is a mainframe system that produces a daily file (e.g., a CSV file) with parcel and route information.
- b. The system must be able to handle hundreds of thousands of status updates per day from thousands of delivery agents across the country.

Exercise:

1. Propose an architecture for the LankaPost Express platform.
2. List and briefly describe the architectural patterns used in the proposed architecture.
3. Based on the description, identify the key legacy process and discuss a method to integrate it.
4. List the high-level sub-systems of the proposed architecture.
5. Identify and describe an interaction/integration in the architecture where Asynchronous Communication is essential.

Answer for Scenario 3: LankaPost Express 📦

1. Proposed Architecture for LankaPost Express

a. Layers/Tiers of the Architecture:

1. **Client Layer:** The applications used by customers and delivery agents.
2. **API Gateway Layer:** Manages and secures all incoming traffic.
3. **Real-time Ingestion Layer:** Handles the high-volume stream of status updates.
4. **Business Logic Layer (Services):** Contains the core logic for tracking and management.
5. **Integration Layer:** Interfaces with the legacy mailsorting system.
6. **Data Layer:** Provides data persistence and caching.

b. Components of Each Layer/Tier:

- **Client Layer:**
 - Customer Web Portal
 - Customer Mobile App (iOS/Android)
 - Delivery Agent Mobile App
- **API Gateway Layer:**
 - **API Gateway:** Routes standard requests (e.g., user login).
 - **WebSocket Gateway:** Manages persistent connections to customer apps for pushing real-time tracking updates.
- **Real-time Ingestion Layer:**
 - **Event Ingestor Endpoint:** A highly scalable endpoint to receive status updates from agent apps.
 - **Message Broker (e.g., Apache Kafka):** A high-throughput message bus to queue incoming status update events.
- **Business Logic Layer (Services):**
 - **Tracking Service:** Consumes status updates from the message broker, updates the parcel's history, and pushes updates to customers.
 - **Parcel Management Service:** Manages parcel information, scheduling, and proof-of-delivery.
 - **User Management Service:** Manages customer and agent accounts.
- **Integration Layer:**
 - **Batch Ingestion Service:** A scheduled service that runs daily to parse the CSV file from the legacy Mailsorting System and populate the parcel database.
- **Data Layer:**
 - **Parcel Database:** Stores all information about parcels, including their full tracking history.
 - **Real-time Status Cache (e.g., Redis):** Stores the *current* status and location of active parcels for extremely fast lookups.

c. Overall Architecture Description:

The process starts when the Batch Ingestion Service imports the day's parcel data from the legacy system's file. When a delivery agent scans a parcel, their mobile app sends a status update to the Event Ingestor Endpoint, which immediately places the event onto a Kafka message queue. The Tracking Service, a subscriber to this queue, processes the event. It updates the parcel's history in the main Parcel

Database and also updates the parcel's latest status in the Redis Cache. Simultaneously, it pushes this new status through the WebSocket Gateway to any customer who is actively tracking that parcel on the web or mobile app, providing an instant update.

2. Architectural Patterns Used

- **Event-Driven Architecture:** The system is built around producing and consuming events (e.g., ParcelScanned, DeliveryCompleted). This is perfect for a real-time tracking system and allows for massive scalability and decoupling of services.
- **Publish-Subscribe Pattern:** Implemented using Kafka. The agent apps "publish" status events, and various backend services "subscribe" to these events to perform their tasks (e.g., updating databases, sending notifications). This allows multiple services to react to the same event independently.
- **CQRS (Command Query Responsibility Segregation):** The system has a clear separation between "Commands" (the high volume of incoming status updates) and "Queries" (customers checking the status). The write path is optimized with Kafka and a durable database, while the read path is optimized with a fast in-memory cache (Redis).
- **API Gateway Pattern:** Provides a secure and managed entry point for all clients⁷.

3. Legacy System Integration

- **Legacy Process Identified:** The key legacy process is the **internal Mailsorting System** which generates a daily CSV file containing parcel and routing information. It's not a real-time system but a batch-oriented one.
- **Method of Integration:** A **Batch File Processing** integration method is required.
 1. **Scheduled Job:** A cron job or a scheduled task is set up to run daily (e.g., at 5 AM), after the legacy system has finished generating its file.
 2. **File Reader Service:** This automated service accesses the location where the CSV file is dropped (e.g., an FTP server or shared directory).
 3. **Parse and Transform:** The service reads the file row by row, parsing the data and transforming it into the modern format used by the LankaPost Express platform's database.
 4. **Bulk Load:** The transformed data is then bulk-inserted into the **Parcel Database**. This pre-populates the system with all the parcels that are scheduled for delivery on that day. This approach completely decouples the new real-time platform from the availability of the old mainframe system. The new system simply depends on a file being present at a specific time.

4. High-Level Sub-Systems

- **Parcel Tracking & Tracing System:** The core real-time system that ingests updates and provides tracking information to customers.

- **Delivery Operations Management:** The mobile application and backend services used by delivery agents.
- **Customer Management System:** Manages customer accounts, login, and profiles.
- **Legacy Data Ingestion System:** The batch subsystem responsible for integrating data from the Mailsorting System.

5. Asynchronous Communication Scenario

- **Interaction/Integration:** The entire flow of **submitting a parcel status update** from the delivery agent's mobile app.
 - **Justification:** This interaction is the heart of the system and must be asynchronous to function at scale.
 - **Responsiveness & Offline Capability:** When an agent scans a QR code, the app sends the update to the server. An asynchronous "fire-and-forget" call means the app doesn't have to wait for the database to be updated. It gets an immediate acknowledgment that the event was received by the message broker. This makes the agent's app feel instantaneous⁸. It also allows the app to queue updates locally if it loses internet connectivity and send them in a batch when it comes back online.
 - **Scalability & Throughput:** Thousands of agents could be scanning parcels at the exact same time. A synchronous system would try to process each request one by one, creating a massive bottleneck. A message broker like Kafka is designed to ingest millions of events per second, acting as a buffer. The backend services can then consume and process these events at a steady, manageable pace, ensuring the system remains stable even under extreme load.
-

Enhanced Answers for Software Architecture Scenarios

Scenario 0: SriToysRus Ltd 🦊

1. Proposed Architecture

A **Microservices Architecture** is proposed to build a scalable and maintainable online platform. This architecture decouples major business functionalities into independent services while maintaining clear separation of concerns.

a. Layers/Tiers of the Architecture:

1. **Client Tier:** User-facing applications providing the interface for customers
2. **API Gateway Tier:** Centralized entry point managing authentication, routing, and cross-cutting concerns
3. **Business Services Tier:** Independent microservices handling specific business domains
4. **Data & Integration Tier:** Manages data persistence and external system communication

b. Components of Each Layer/Tier:

Client Tier:

- **Web Application:** Customer-facing e-commerce site with responsive design
- **Mobile Application (Future):** iOS/Android native apps for enhanced user experience

API Gateway Tier:

- **API Gateway:** Central routing hub managing authentication, rate limiting, and request distribution
- **Authentication Service:** Dedicated service for user registration, login, and session management

Business Services Tier:

- **Product Service:** Manages toy catalog, search functionality, detailed descriptions, and categorization
- **Order Service:** Handles shopping cart operations, checkout process, order history, and transaction management
- **User Service:** Manages customer profiles, addresses, wish lists, and personalization features
- **Inventory Service:** Real-time stock management with legacy system synchronization

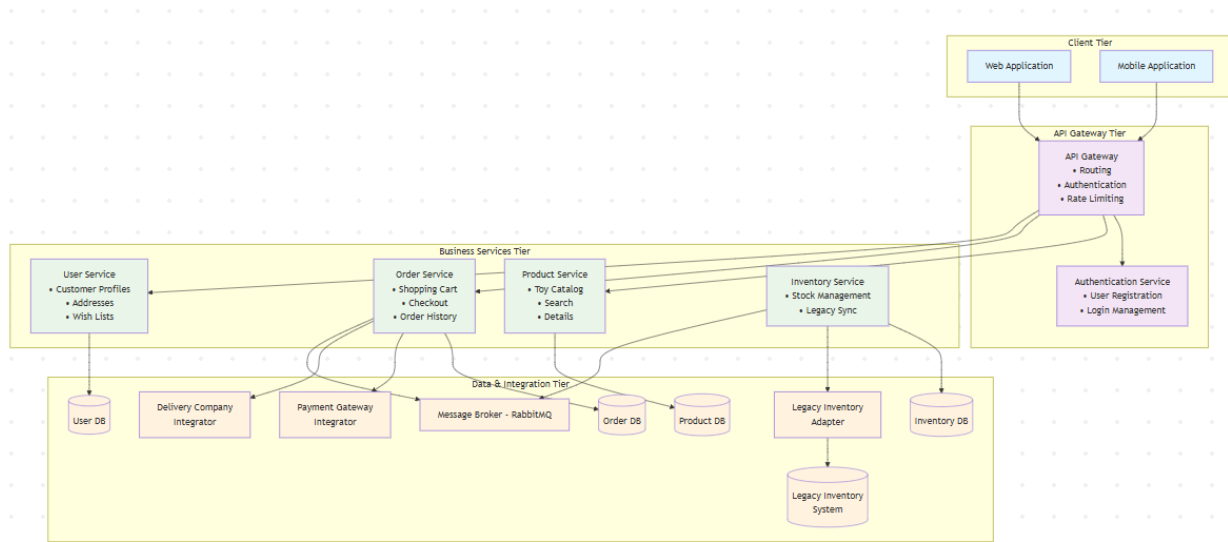
Data & Integration Tier:

- **Databases:** Domain-specific databases (Product DB, Order DB, User DB, Inventory DB) ensuring loose coupling
- **Legacy Inventory Adapter:** Modernizes legacy system interface with RESTful API wrapper

- **Payment Gateway Integrator:** Secure integration with external payment processors
- **Delivery Company Integrator:** Interface with delivery tracking platforms
- **Message Broker (RabbitMQ):** Enables asynchronous communication between services

c. Architecture Description:

The architecture follows a layered approach where customers interact through web/mobile applications that communicate exclusively with the API Gateway. The gateway authenticates users and intelligently routes requests to appropriate microservices. For instance, product browsing hits the Product Service, cart operations involve the Order Service, and checkout coordinates between Order, Inventory, Payment, and Delivery services. The Message Broker enables resilient asynchronous operations, particularly for inventory updates and order processing workflows.



2. Architectural Patterns

- **Microservices Architecture:** Decomposes the system into fine-grained, independently deployable services. Each service owns its data and can be developed, deployed, and scaled independently by different teams.
- **API Gateway Pattern:** Provides a single, secure entry point for all client requests. Centralizes cross-cutting concerns like authentication, logging, monitoring, rate limiting, and request/response transformation.
- **Adapter Pattern:** Wraps the legacy inventory system and third-party APIs (payment gateway, delivery company) to provide standardized interfaces that integrate seamlessly with modern microservices.
- **Event-Driven Architecture:** Uses message broker for asynchronous communication, enabling loose coupling and improved system resilience.

3. Legacy System Integration

Legacy System Identified: The existing "computerized inventory management system that links all distributed outlets."

Integration Method: Hybrid approach combining synchronous and asynchronous patterns:

1. **Real-time Stock Queries (Synchronous):**

- Legacy Inventory Adapter exposes REST API (e.g., GET /stock/{product_id})
- Product Service calls adapter for real-time stock verification during browsing
- Ensures accurate availability display to customers

2. **Inventory Updates (Asynchronous):**

- Order Service publishes ProductSold events to Message Broker upon successful checkout
- Inventory Service subscribes to these events and updates legacy system
- Provides fast checkout experience while ensuring eventual consistency

3. **Data Transformation:** Adapter handles protocol translation between modern REST APIs and legacy system interfaces

4. **High-Level Sub-Systems**

- **Product Catalog Management:** Comprehensive toy information management including categories, search, filtering, and detailed product descriptions
- **Customer Experience Management:** User accounts, personalization, wish lists, and purchase history
- **Order Processing & Fulfillment:** End-to-end order management from cart to delivery
- **Inventory Synchronization System:** Real-time stock management across online and physical channels
- **External Integration Platform:** Payment processing and delivery management services

5. **Asynchronous Communication Scenario**

Interaction: Post-Purchase Order Processing Workflow

After successful payment confirmation, multiple operations must occur: inventory updates, email confirmations, and delivery scheduling.

Asynchronous Implementation:

1. Order Service receives payment confirmation and immediately responds to customer with "Thank You" page
2. Simultaneously publishes OrderPlaced event to Message Broker
3. Multiple services process the event in parallel:

- **Inventory Service:** Decrements stock in legacy system
- **Notification Service:** Sends confirmation email to customer
- **Delivery Integration Service:** Schedules delivery with external company

Benefits: Faster user experience, improved system resilience (failed email doesn't affect order), and enhanced scalability through parallel processing.

Scenario 1: HealthConnect Patient Portal

1. Proposed Architecture

A **Service-Oriented Architecture (SOA)** with microservices characteristics, specifically designed for healthcare data integration, security compliance, and interoperability with legacy systems.

a. Layers/Tiers of the Architecture:

1. **Client Tier:** Patient and staff-facing applications
2. **Presentation Tier / API Gateway:** Secure entry point with healthcare-specific security measures
3. **Business/Service Tier:** Healthcare domain-specific microservices
4. **Integration/Data Tier:** Legacy system integration and data management

b. Components of Each Layer/Tier:

Client Tier:

- **Web Application:** Responsive web portal for cross-platform access
- **Mobile Application:** Native iOS/Android apps with offline capabilities and push notifications

Presentation Tier / API Gateway:

- **API Gateway:** Healthcare-compliant routing with audit logging and security enforcement
- **Authentication Service:** Multi-factor authentication with role-based access control (RBAC)

Business/Service Tier:

- **Patient Profile Service:** Unified patient identity management with data consolidation from multiple sources
- **Appointment Service:** Multi-hospital scheduling with conflict resolution and availability management
- **Billing Service:** Integrated payment processing with insurance claim management
- **Lab Report Service:** Secure diagnostic report management with viewing permissions
- **Notification Service:** HIPAA-compliant communication (email, SMS, push notifications)

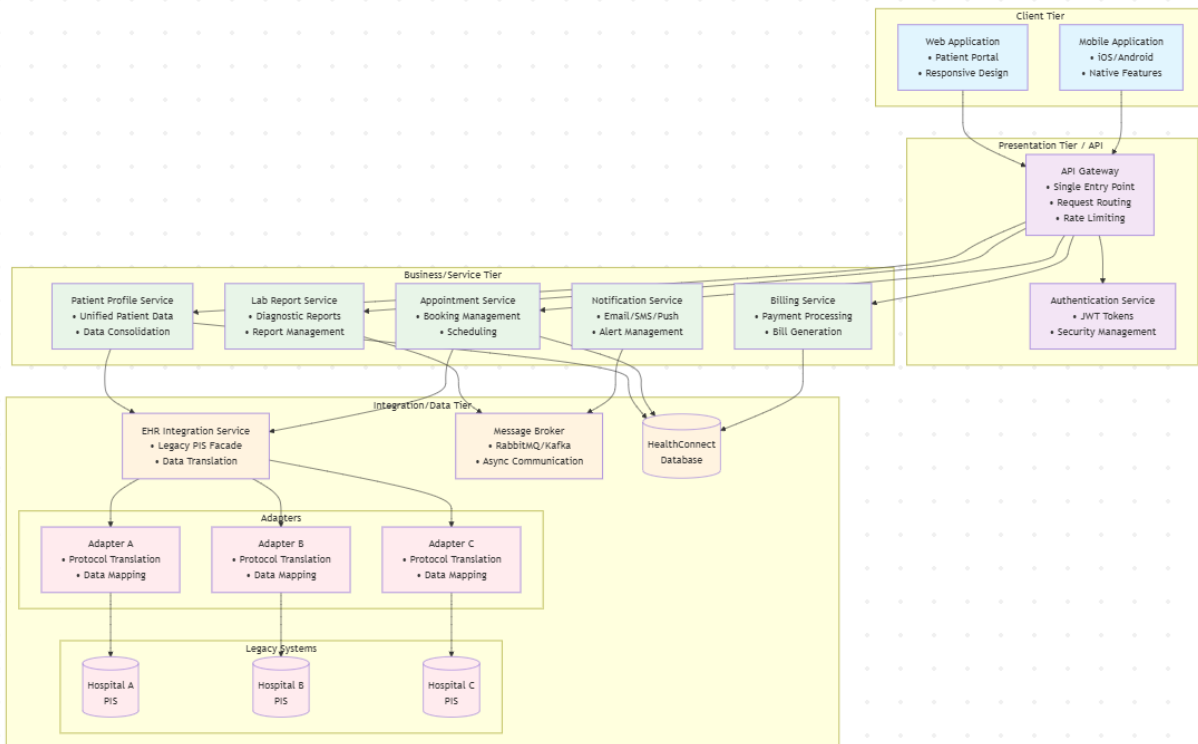
Integration/Data Tier:

- **EHR Integration Service:** Sophisticated facade managing communication with heterogeneous legacy systems
- **Message Broker (RabbitMQ/Kafka):** Healthcare event processing with guaranteed delivery
- **HealthConnect Database:** Central repository for unified platform data

- **Legacy PIS Adapters:** Hospital-specific adapters for protocol and data format translation

c. Architecture Description:

The architecture provides a unified healthcare experience while respecting the complexity of existing hospital infrastructure. The EHR Integration Service acts as an intelligent mediator, querying multiple legacy Patient Information Systems, transforming disparate data formats into a canonical model, and presenting a consolidated patient view. The system ensures HIPAA compliance through encrypted communication, audit trails, and role-based access controls.



2. Architectural Patterns

- **Facade Pattern:** The EHR Integration Service provides a simplified interface to complex legacy systems, hiding implementation details from client services.
- **Adapter Pattern:** Each legacy PIS has a dedicated adapter that translates between modern REST APIs and proprietary legacy protocols, enabling seamless integration.
- **Canonical Data Model:** Standardized data representation across all services ensures consistent patient information regardless of source system.
- **Event-Driven Architecture:** Asynchronous processing for non-critical operations like notifications and reporting.

3. Legacy System Integration

Primary Challenge: Multiple heterogeneous Patient Information Systems across hospital branches with different databases, schemas, and communication protocols.

Integration Strategy:

1. **Canonical Data Model Definition:** Establish standardized patient data schema (JSON structure for "Patient Record," "Medical History," etc.)
2. **Adapter Development:** Create hospital-specific adapters that:
 - Translate requests from canonical format to legacy system queries
 - Transform legacy responses back to standardized format
 - Handle different authentication and connection protocols
3. **EHR Integration Service:** Central orchestration service that:
 - Receives unified requests from business services
 - Distributes queries to appropriate hospital adapters
 - Aggregates and consolidates responses
 - Provides caching for frequently accessed data
4. **Error Handling:** Graceful degradation when specific hospital systems are unavailable

4. High-Level Sub-Systems

- **Patient Identity & Profile Management:** Unified patient records with multi-source data consolidation
- **Healthcare Scheduling System:** Cross-hospital appointment management with resource optimization
- **Clinical Information Management:** Lab reports, medical history, and prescription management
- **Financial Services Integration:** Billing, insurance, and payment processing with legacy system connectivity

5. Asynchronous Communication Scenario

Interaction: Lab Report Availability Notification

When a new lab report is uploaded to any hospital's PIS, patients should be notified automatically without impacting system performance.

Asynchronous Implementation:

1. Lab Report Service detects new report through scheduled polling or webhook
2. Publishes LabReportAvailable event to Message Broker with patient ID and report metadata
3. Notification Service subscribes to these events and processes them independently
4. Generates appropriate notifications (email, SMS, push) based on patient preferences

5. Maintains delivery status and retry logic for failed notifications

Healthcare Benefits:

- Immediate patient notification improves care coordination
- System remains responsive during high-volume report processing
- Failed notifications don't impact core medical workflows
- Audit trail for compliance requirements

Scenario 2: MetroLink Transit System 🚊

1. Proposed Architecture

An **Event-Driven Architecture** optimized for real-time GPS data processing, high-concurrency user requests, and scalable message handling to support large-scale public transportation systems.

a. Layers/Tiers of the Architecture:

1. **Client Tier:** Commuter-facing mobile applications
2. **API Gateway Tier:** Dual-gateway approach for different communication patterns
3. **Real-time Processing Tier:** High-throughput data ingestion and processing
4. **Service Tier:** Business logic for transit operations
5. **Data Tier:** Optimized storage for both real-time and static data

b. Components of Each Layer/Tier:

Client Tier:

- **MetroLink Mobile App:** Cross-platform application with real-time tracking, trip planning, and digital wallet functionality

API Gateway Tier:

- **API Gateway:** Handles standard REST requests (authentication, trip planning, wallet operations)
- **WebSocket Gateway:** Manages persistent connections for real-time location broadcasting to thousands of concurrent users

Real-time Processing Tier:

- **Data Ingestor:** Horizontally scalable endpoint capable of receiving GPS updates from entire bus fleet
- **Message Broker (Apache Kafka):** High-throughput message streaming platform with partitioning for scalability
- **Location Processor Service:** Intelligent GPS data processing including route mapping, speed calculation, and arrival prediction

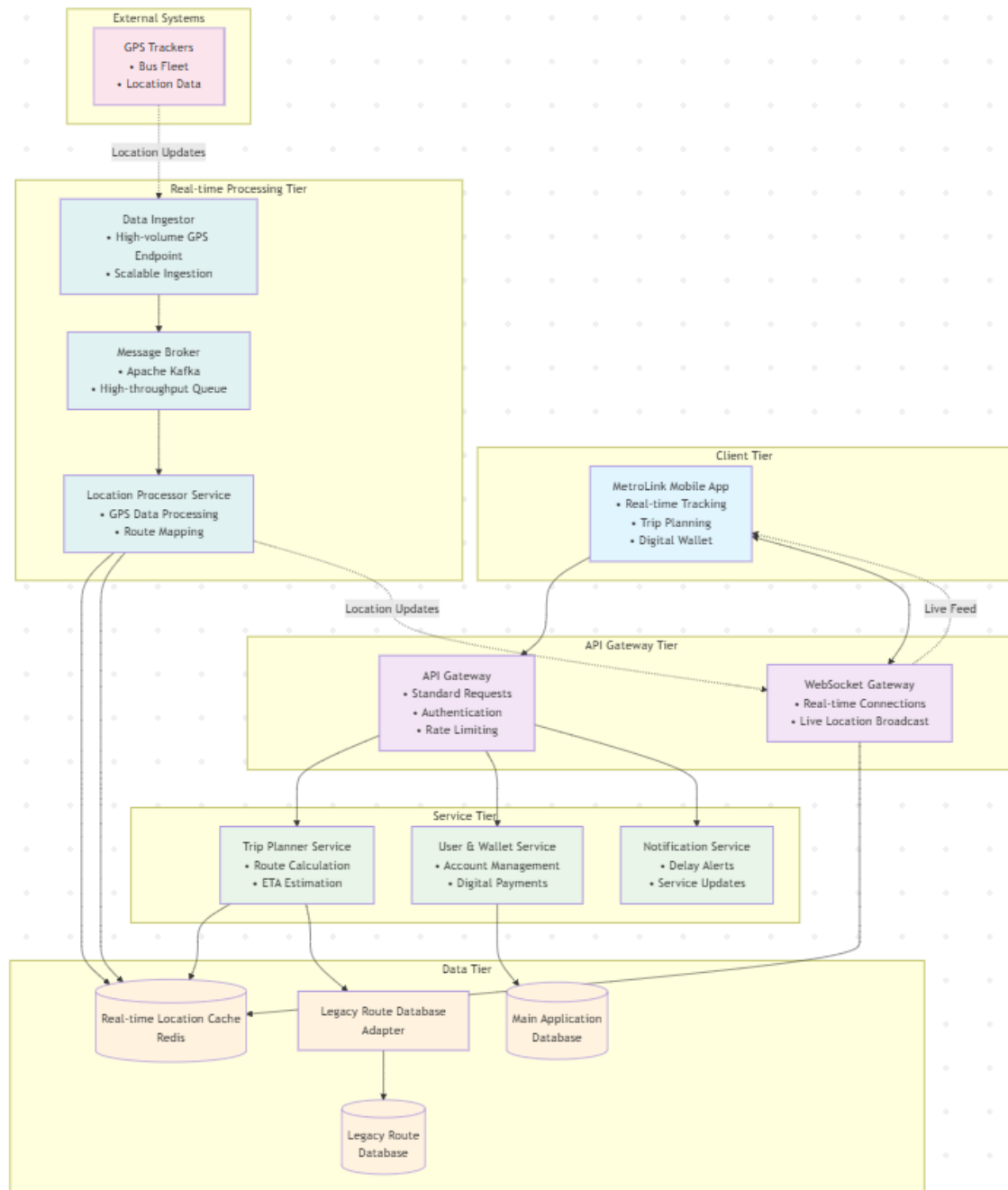
Service Tier:

- **Trip Planner Service:** Advanced route calculation using real-time data, static schedules, and traffic patterns
- **User & Wallet Service:** Account management with secure digital payment processing
- **Notification Service:** Proactive alerts for delays, route changes, and service disruptions

Data Tier:

- **Legacy Route Database Adapter:** Wrapper providing modern API access to existing route/schedule data
- **Real-time Location Cache (Redis):** Ultra-fast in-memory storage for current bus positions and status
- **Main Application Database:** Persistent storage for user data, transaction history, and analytics

c. Architecture Description:



The system processes continuous GPS streams from the bus fleet through a scalable ingestion pipeline. GPS data flows through Kafka topics to Location Processor Services that update Redis cache with current positions. The WebSocket Gateway maintains live connections with mobile clients, broadcasting location updates in real-time. Trip planning combines static route data with live positions to provide accurate ETAs. The architecture scales horizontally to handle peak usage periods.

2. Architectural Patterns

- **Event-Driven Architecture:** Core pattern enabling real-time processing of GPS events with loose coupling between producers and consumers.
- **Publish-Subscribe Pattern:** Kafka implementation allows multiple services to consume GPS events independently (location updates, analytics, maintenance alerts).
- **CQRS (Command Query Responsibility Segregation):** Separates high-volume GPS writes (optimized for throughput) from user queries (optimized for response time).
- **Circuit Breaker Pattern:** Protects against legacy system failures, ensuring real-time features continue operating even when static data is unavailable.

3. Legacy System Integration

Legacy System: Existing database containing static bus routes, schedules, and fare information without modern API access.

Integration Method - ETL with Caching:

1. **Scheduled Synchronization Service:** Automated process running at regular intervals (hourly/daily)
2. **Extract:** Connects to legacy database using appropriate drivers/protocols
3. **Transform:** Converts legacy data formats to modern JSON schemas optimized for mobile consumption
4. **Load:** Updates both Main Application Database and Redis cache for fast access
5. **Decoupling Strategy:** Real-time system never directly queries legacy database, ensuring performance isolation

Benefits: Legacy system performance doesn't impact real-time operations, enables gradual migration, and provides fallback capabilities.

4. High-Level Sub-Systems

- **Real-time Vehicle Tracking Platform:** GPS ingestion, processing, and live location broadcasting
- **Intelligent Trip Planning Engine:** Multi-modal route calculation with real-time optimization
- **Digital Payment & Fare Management:** Secure wallet operations with dynamic fare calculation
- **Passenger Communication System:** Proactive notifications and service alerts

5. Asynchronous Communication Scenario

Interaction: GPS Location Data Processing Pipeline

The entire GPS tracking workflow represents the core asynchronous communication pattern essential for system functionality.

Implementation:

1. **GPS Data Ingestion:** Bus trackers send location updates via fire-and-forget HTTP posts to Data Ingestor
2. **Event Streaming:** Ingestor immediately publishes events to Kafka topics without waiting for processing completion
3. **Parallel Processing:** Multiple Location Processor instances consume events concurrently, enabling horizontal scaling
4. **Real-time Updates:** Processed data updates Redis cache and triggers WebSocket broadcasts to connected mobile clients
5. **Analytics Pipeline:** Same events feed separate analytics services for route optimization and performance monitoring

Critical Benefits:

- **Scalability:** System handles thousands of simultaneous GPS updates without bottlenecks
- **Resilience:** Failed processing of individual GPS points doesn't affect overall system operation
- **Performance:** Mobile apps receive instant location updates without waiting for database writes
- **Offline Capability:** Message buffering ensures no data loss during temporary connectivity issues

Scenario 3: UniPortal University Management System 🎓

1. Proposed Architecture

A **Microservices Architecture** with specialized load management capabilities designed to handle extreme traffic variations, particularly during course registration periods, while integrating multiple legacy university systems.

a. Layers/Tiers of the Architecture:

1. **Presentation Layer:** Student and administrative web interfaces
2. **Service Layer (API Gateway):** Intelligent traffic management with peak-load optimization
3. **Business Logic Layer:** Domain-specific microservices for university operations
4. **Integration Layer:** Legacy system adapters with standardized interfaces
5. **Data Access Layer:** Optimized data persistence and caching strategies

b. Components of Each Layer/Tier:

Presentation Layer:

- **Student Web Portal:** Modern React-based interface for course registration, grade viewing, and profile management
- **Admin/Staff Portal:** Administrative interface for academic management and system oversight

Service Layer (API Gateway):

- **API Gateway:** Intelligent request routing with load balancing and service discovery
- **Authentication & Session Management:** JWT-based authentication with session clustering for high availability
- **Rate Limiting & Queue Management:** Advanced throttling mechanisms for registration period traffic control

Business Logic Layer (Microservices):

- **Student Profile Service:** Unified student identity with academic history aggregation
- **Course & Enrollment Service:** Sophisticated registration system with conflict detection and capacity management
- **Grades Service:** Comprehensive grade management with transcript generation and GPA calculation
- **Notification Service:** Multi-channel communication system (email, SMS, push notifications)

Integration Layer:

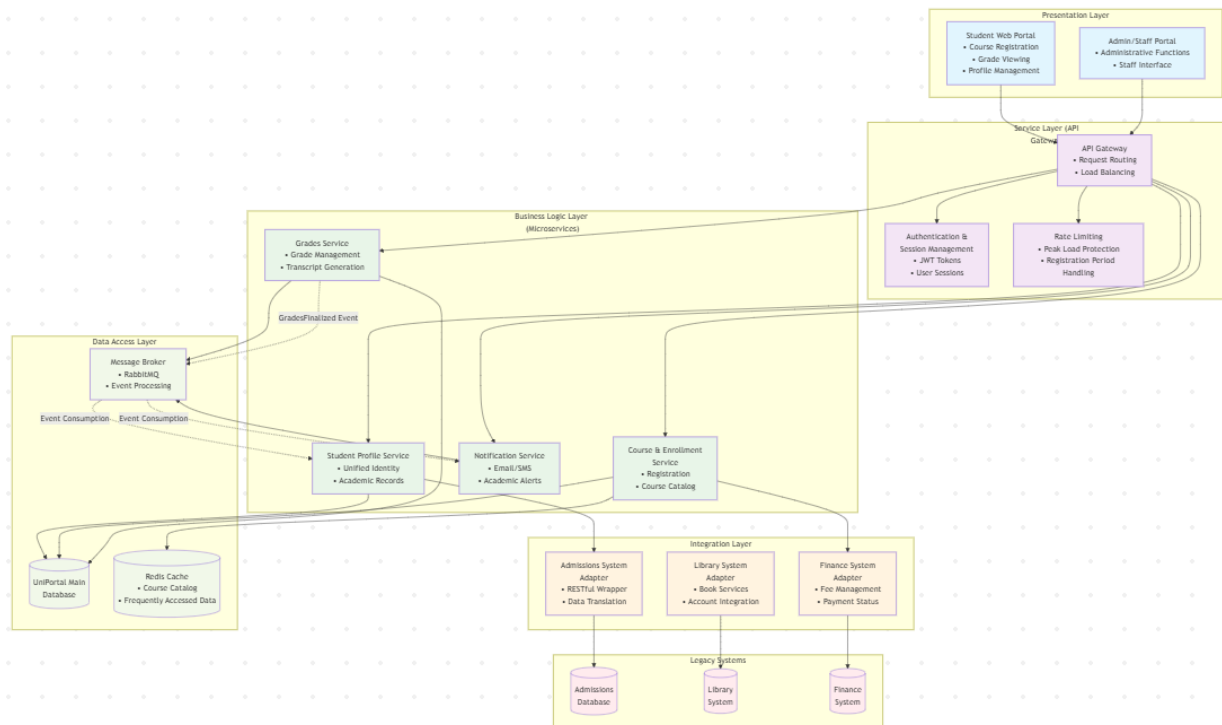
- **Admissions System Adapter:** RESTful wrapper for legacy admissions database with data validation
- **Library System Adapter:** Integration for library services including book searches and account management
- **Finance System Adapter:** Financial clearance verification and fee management integration

Data Access Layer:

- **UniPortal Main Database:** Primary data store with optimized schemas for university operations
- **Redis Cache Cluster:** High-performance caching for course catalogs and frequently accessed data
- **Message Broker (RabbitMQ):** Event-driven communication with guaranteed message delivery

c. Architecture Description:

The architecture prioritizes scalability and resilience, particularly during high-traffic registration periods. The Rate Limiting component implements intelligent queuing to prevent system overload while maintaining fair access. Microservices communicate through well-defined APIs and asynchronous events. Legacy integration adapters provide clean, modern interfaces to aging university systems while maintaining data consistency across platforms.



2. Architectural Patterns

- **Microservices Architecture:** Domain-driven service decomposition enabling independent development, deployment, and scaling of university functions.

- **API Gateway Pattern:** Centralized entry point providing authentication, rate limiting, request routing, and monitoring capabilities.
- **Adapter Pattern:** Legacy system integration through standardized wrappers that translate between modern REST APIs and proprietary university system interfaces.
- **Event-Driven Architecture:** Asynchronous processing for grade submissions, notifications, and academic workflow automation.
- **CQRS with Event Sourcing:** Course registration system separates command processing (enrollment requests) from query operations (seat availability) for optimal performance during peak loads.

3. Legacy System Integration

Legacy Systems: Admissions System, Library Management System, and Finance System with proprietary databases and interfaces.

Integration Strategy - API Wrapper Pattern:

1. **Canonical Data Model:** Define standard university data schemas (Student, Course, Financial Status, etc.) for platform consistency
2. **Adapter Development:** Create microservice adapters for each legacy system:
 - **Admissions Adapter:** Exposes GET /students/{id}/admission-status wrapping complex legacy queries
 - **Finance Adapter:** Provides GET /students/{id}/financial-clearance with real-time fee status
 - **Library Adapter:** Offers GET /students/{id}/library-account for borrowed books and fines
3. **Data Translation:** Adapters handle:
 - Protocol conversion (database connections to REST APIs)
 - Data format transformation (legacy formats to JSON)
 - Error handling and timeout management
4. **Caching Strategy:** Frequently accessed legacy data cached in Redis to reduce load on aging systems

4. High-Level Sub-Systems

- **Academic Lifecycle Management:** Student admissions, enrollment, progression tracking, and graduation processing
- **Course Management & Registration:** Comprehensive course catalog, capacity management, and intelligent registration workflows

- **Financial Integration Platform:** Fee management, payment processing, and financial clearance verification
- **Library Services Integration:** Book searches, account management, and resource access control
- **Communication & Notification Engine:** Multi-channel messaging for academic alerts and administrative communications

5. Asynchronous Communication Scenario

Interaction: Grade Submission and Academic Processing Workflow

When faculty submit final grades, multiple downstream processes must execute without impacting the submission experience.

Asynchronous Implementation:

1. **Grade Submission:** Faculty submits grades through Grades Service
2. **Immediate Response:** System validates and stores grades, immediately confirming successful submission
3. **Event Publication:** Grades Service publishes GradesFinalized event to RabbitMQ with course and student details
4. **Parallel Processing:** Multiple services consume the event independently:
 - **Notification Service:** Sends grade notification emails to all enrolled students
 - **Student Profile Service:** Recalculates semester GPA and updates academic standing
 - **Analytics Service:** Updates course performance metrics and academic reporting

Academic Benefits:

- **Faculty Experience:** Immediate confirmation of grade submission without waiting for all processing
- **System Resilience:** Failed email delivery doesn't impact grade recording
- **Scalability:** Parallel processing handles large courses efficiently
- **Audit Trail:** Event-driven architecture provides complete academic workflow tracking for compliance

Scenario 4: LankaPost Express Courier System 📦

1. Proposed Architecture

A **Hybrid Event-Driven Architecture** combining real-time status tracking with batch legacy integration, designed to handle massive volumes of parcel status updates while maintaining customer-facing real-time visibility.

a. Layers/Tiers of the Architecture:

1. **Client Layer:** Multi-stakeholder applications for customers and delivery personnel
2. **API Gateway Layer:** Dual-gateway architecture for different communication patterns
3. **Real-time Ingestion Layer:** High-volume status update processing pipeline
4. **Business Logic Layer:** Core courier service operations
5. **Integration Layer:** Legacy system integration with batch processing
6. **Data Layer:** Optimized storage for both transactional and analytical workloads

b. Components of Each Layer/Tier:

Client Layer:

- **Customer Web Portal:** Comprehensive tracking interface with delivery scheduling and history
- **Customer Mobile App:** Real-time tracking with push notifications and delivery preferences
- **Delivery Agent Mobile App:** QR code scanning, status updates, and route optimization

API Gateway Layer:

- **API Gateway:** Standard REST API management for authentication and business operations
- **WebSocket Gateway:** Real-time communication channel for instant tracking updates to customers

Real-time Ingestion Layer:

- **Event Ingestor Endpoint:** Massively scalable endpoint designed for hundreds of thousands of daily status updates
- **Message Broker (Apache Kafka):** High-throughput message streaming with partitioning by geographic regions

Business Logic Layer:

- **Tracking Service:** Intelligent status processing with validation, history management, and real-time customer notifications

- **Parcel Management Service:** Comprehensive parcel lifecycle management including scheduling and proof-of-delivery
- **User Management Service:** Customer accounts, delivery preferences, and agent authentication

Integration Layer:

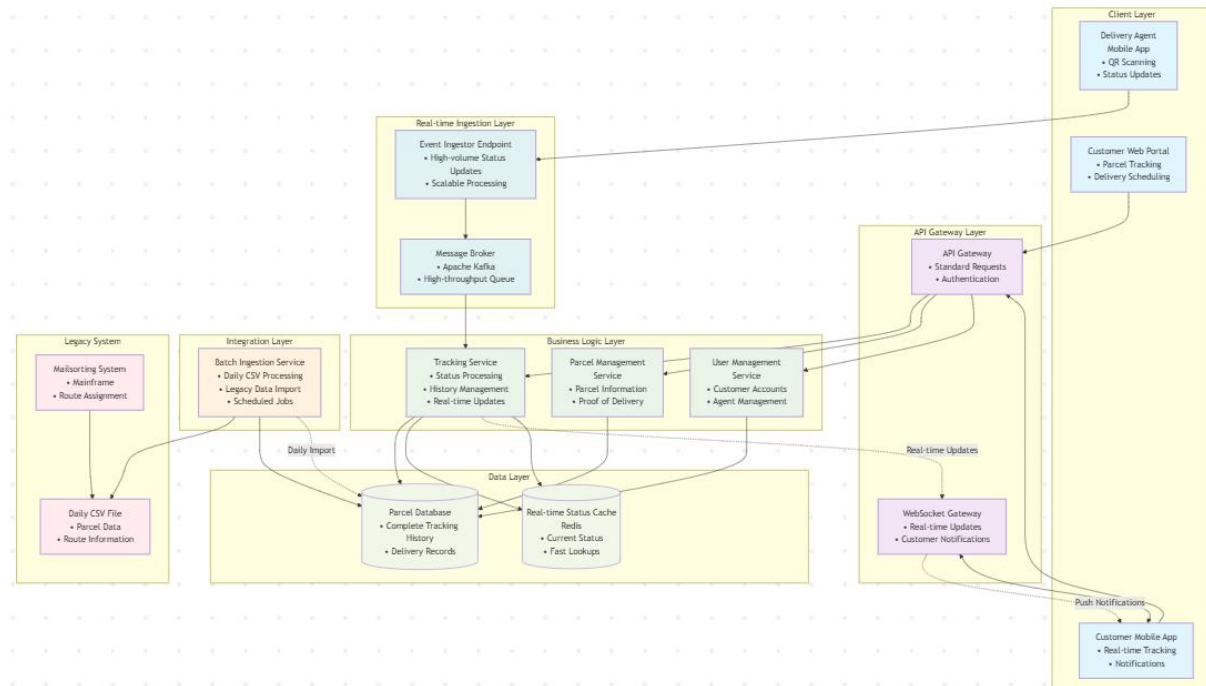
- **Batch Ingestion Service:** Automated daily processing of CSV files from legacy mailsorting system with error handling and validation

Data Layer:

- **Parcel Database:** Complete audit trail of all parcel movements with geographic indexing
- **Real-time Status Cache (Redis):** Ultra-fast lookup for current parcel status and location information

c. Architecture Description:

The system operates on dual data flows: batch import of daily parcel assignments from the legacy mainframe system, and real-time status updates from field agents. The Batch Ingestion Service processes the daily CSV file, populating the system with new parcels and routing information. Throughout the day, delivery agents scan QR codes, sending status updates through the high-volume Event Ingestor. These updates flow through Kafka to the Tracking Service, which updates both persistent storage and Redis cache while triggering real-time notifications to customers via WebSocket connections.



2. Architectural Patterns

- **Event-Driven Architecture:** Core pattern enabling real-time processing of status updates with loose coupling between components.

- **CQRS (Command Query Responsibility Segregation):** Separates high-volume status writes (optimized for agent operations) from customer tracking queries (optimized for fast response).
- **Publish-Subscribe Pattern:** Kafka enables multiple services to process status events independently (tracking updates, analytics, customer notifications).
- **Batch Processing Pattern:** ETL pipeline for legacy system integration with daily data synchronization.
- **Circuit Breaker Pattern:** Protects against legacy system failures, ensuring real-time tracking continues even during mainframe downtime.

3. Legacy System Integration

Legacy System: Mainframe-based Mailsorting System generating daily CSV files with parcel and routing information.

Integration Method - Automated Batch Processing:

1. **Scheduled File Processing:** Automated service runs daily at 5 AM after legacy system completes file generation
2. **File Acquisition:** Service monitors designated file location (FTP server or shared directory) for new CSV files
3. **Data Validation:** Comprehensive validation including format verification, duplicate detection, and business rule compliance
4. **Transformation Pipeline:** Convert legacy data formats to modern database schemas with geographic indexing
5. **Bulk Database Operations:** Efficient batch insertion of parcel records with transaction management
6. **Error Handling:** Robust exception management with alerts for processing failures and data quality issues

Decoupling Benefits: Real-time tracking operates independently of legacy system availability, enabling continuous service even during mainframe maintenance.

4. High-Level Sub-Systems

- **Real-time Parcel Tracking Platform:** High-volume status ingestion with instant customer visibility
- **Delivery Operations Management:** Mobile-first agent tools with offline capability and route optimization
- **Customer Experience Platform:** Multi-channel tracking interface with proactive notifications
- **Legacy Data Integration System:** Automated batch processing with comprehensive error handling and monitoring

5. Asynchronous Communication Scenario

Interaction: End-to-End Parcel Status Update Processing

The complete workflow from agent QR scan to customer notification represents the core asynchronous communication pattern.

Comprehensive Implementation:

1. **Agent Status Update:** Delivery agent scans QR code, app sends status update to Event Ingestor with timestamp, location, and proof-of-delivery
2. **Immediate Acknowledgment:** Agent receives instant confirmation that update was received, enabling offline queueing capabilities
3. **Event Streaming:** Ingestor publishes event to Kafka topic partitioned by geographic region for scalability
4. **Parallel Processing:** Multiple Tracking Service instances consume events concurrently:
 - Update complete parcel history in Parcel Database
 - Refresh current status in Redis cache for fast customer queries
 - Trigger real-time notifications through WebSocket Gateway
5. **Customer Notification:** Connected customers receive instant updates via WebSocket, while others get push notifications
6. **Analytics Processing:** Same events feed analytics pipeline for delivery performance monitoring and route optimization

Operational Benefits:

- **Agent Productivity:** Instant acknowledgment enables rapid scanning without waiting for database updates
- **Customer Satisfaction:** Real-time visibility with sub-second update delivery
- **System Resilience:** Message buffering ensures no status updates are lost during peak periods or system maintenance
- **Scalability:** Kafka partitioning enables horizontal scaling across geographic regions
- **Offline Capability:** Agents can continue scanning with local queueing when connectivity is poor